

XOR Linked List Simulator

Linked Lists, XOR (Memory-Optimized) Linked List · Mentor-Led Lab

Developed by TalenciaGlobal

Difficulty ★★★★★ Advanced

Problem

A doubly linked list normally stores two pointers per node. An XOR linked list stores just one: the XOR of the previous and next addresses, halving the link memory. Because high-level languages hide raw addresses, you will simulate it with node indices.

Given a list of values, build the XOR linked list, then traverse it forwards and backwards using only the single XOR link per node, and print the stored link field.

What to Build

- Index the nodes 1..n, using 0 to mean null. Each node i stores $\text{link}[i] = \text{prevIndex} \text{ XOR } \text{nextIndex}$.
- Forward traversal: keep prev and cur; the next index is $\text{link}[\text{cur}] \text{ XOR } \text{prev}$.
- Backward traversal: start from the tail and apply the same XOR step.
- Print the forward order, the backward order, and the link field of every node.

Input / Output Format

Input: Line 1: n . Line 2: n values.

Output: Three lines: Forward, Backward, XOR Links.

Examples

Example 1

Input: $n = 5$, values = 10 20 30 40 50

Output:

```
Forward: 10 20 30 40 50
Backward: 50 40 30 20 10
XOR Links: 2 2 6 6 4
```

Explanation: $\text{link}[1]=0^2=2$, $\text{link}[2]=1^3=2$, $\text{link}[3]=2^4=6$, $\text{link}[4]=3^5=6$, $\text{link}[5]=4^0=4$; the XOR step walks the list both ways.

Example 2

Input: n = 3, values = 7 8 9

Output:

```
Forward: 7 8 9
Backward: 9 8 7
XOR Links: 2 2 2
```

Explanation: $\text{link}[1] = 0^2 = 2$, $\text{link}[2] = 1^3 = 2$, $\text{link}[3] = 2^0 = 2$.

Constraints

- $1 \leq n \leq 10^5$; node index 0 represents null.
- Use one link field per node (prevIndex XOR nextIndex).
- Traversal must use the XOR trick, not a separate next/prev array.